

Python AD Audit

Vue d'ensemble du projet

Rapport de synthèse généré le 08/09/2026 — à usage interne, pour prise en main rapide du projet.

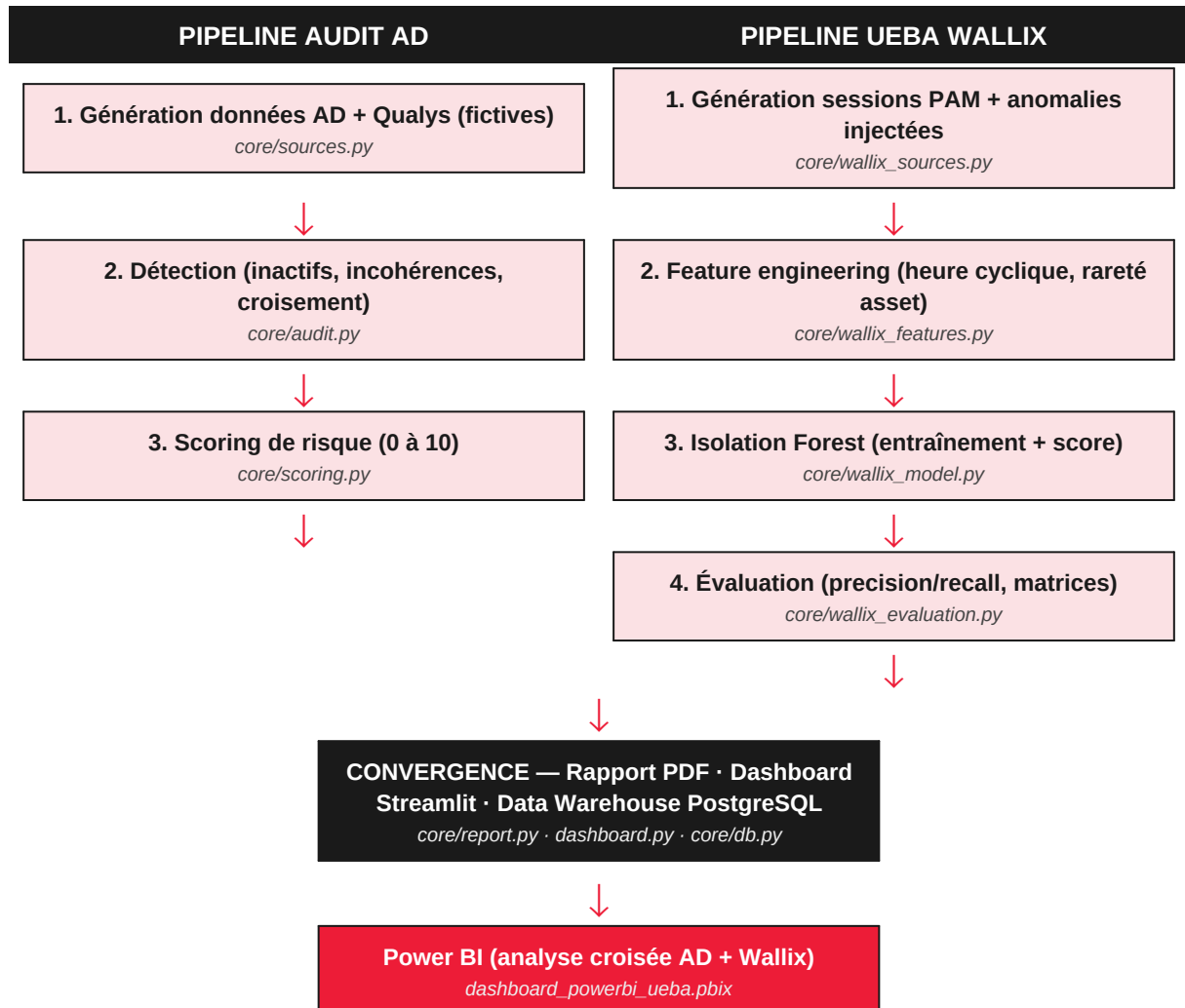
Contexte : simulateur d'audit de sécurité Active Directory développé pendant un stage sécurité/infrastructure en environnement bancaire. En l'absence d'accès à des données réelles, l'ensemble du projet fonctionne sur des données **entièrement fictives** (générées par Faker et par des règles de simulation), mais applique une logique d'audit et de détection réaliste : comptes inactifs, incohérences de droits, corrélation avec des vulnérabilités simulées façon Qualys, et détection d'anomalies comportementales sur des sessions d'accès privilégié (bastion Wallix).

Ce document couvre :

1. L'architecture globale et le flux de données, du bout en bout
2. Le rôle de chaque module du projet
3. La logique du pipeline "Audit AD" (détection + scoring de risque)
4. La logique du pipeline "UEBA Wallix" (Isolation Forest)
5. Le point de convergence : dashboard, rapport PDF, data warehouse & Power BI

1. Architecture globale & flux de données

Le projet est composé de deux pipelines indépendants qui partagent la même logique (générer des données réalistes → les analyser → produire un score de risque), et qui convergent vers les mêmes couches de restitution : dashboard Streamlit, rapport PDF et entrepôt de données PostgreSQL / Power BI.



2. Rôle de chaque module

Le dossier `scripts/` ne contient volontairement aucune logique métier : chaque script `phaseN_*.py` se contente d'appeler ces modules dans le bon ordre (voir section 3). C'est ce qui permet de tester `core/` indépendamment (voir `tests/`) et de réutiliser exactement le même code dans le dashboard.

Fichier	Rôle	Entrée	Sortie
<code>core/config.py</code>	Paramètres centralisés (seuils, poids, chemins)	—	Objet Config utilisé partout
<code>core/sources.py</code>	Génère les données AD et Qualys fictives	Config	DataFrames AD / Qualys + CSV
<code>core/audit.py</code>	Règles de détection (inactifs, incohérences, croisement)	DataFrames AD/Qualys	Alertes
<code>core/scoring.py</code>	Calcule le score de risque combiné (0-10)	Data + alertes	DataFrame scoré
<code>core/report.py</code>	Génère le rapport PDF d'audit AD	DataFrame scoré	PDF (outputs/rapport_audit.pdf)
<code>dashboard.py</code>	Application Streamlit (onglet Audit AD + appel onglet Wallix)	<code>core/*</code> (AD)	Dashboard web
<code>core/wallix_sources.py</code>	Simule sessions PAM Wallix + anomalies injectées	Profils comportementaux	DataFrame sessions
<code>core/wallix_features.py</code>	Transforme les sessions en variables numériques pour le ML	Sessions brutes	Features (hour_sin/cos, rareté asset...)
<code>core/wallix_model.py</code>	Entraîne l'Isolation Forest et score les sessions	Features	Score d'anomalie + prédiction
<code>core/wallix_evaluation.py</code>	Mesure precision/recall/F1, trace les graphiques	Sessions scorées	Métriques + PNG
<code>core/wallix_dashboard.py</code>	Vue Streamlit "Sessions Wallix à risque"	CSV sessions scorées	Onglet dashboard
<code>core/db.py</code>	Construit l'entrepôt de données (star schema)	Résultats AD + Wallix	Tables PostgreSQL

3. Logique du pipeline « Audit AD »

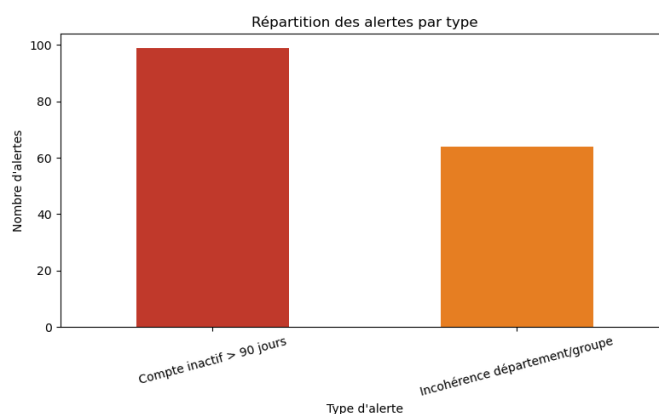
Enchaînement construit progressivement de la Phase 5 à la Phase 8 (scripts/phase5_main.py → phase8_main.py), chaque script ajoutant une brique par rapport au précédent :

1. Générer 200 utilisateurs AD fictifs (Faker) + un scan Qualys simulé (même identifiant id des deux côtés, pour pouvoir croiser).
2. Appliquer les règles de détection : compte inactif > 90 jours, groupe AD incohérent avec le département, ou privilège + machine vulnérable non patchée.
3. Transformer ces alertes en un score chiffré de 0 à 10 par utilisateur (somme pondérée des facteurs détectés, normalisée sur le score maximum théorique).
4. Restituer le résultat : rapport PDF (top 10 des comptes à risque) et/ou dashboard Streamlit interactif (filtres département / niveau / score).

Barème du score de risque

Score	Niveau	Signification
0 – 2	Faible	Aucun facteur de risque significatif détecté
2 – 5	Moyen	Un facteur de risque isolé (ex : compte inactif)
5 – 8	Élevé	Plusieurs facteurs cumulés (ex : privilège + incohérence)
8 – 10	Critique	Cumul de facteurs graves (privilège + vulnérabilité critique non patchée)

Illustration — répartition des alertes générées (scripts/phase4_visualisation.py)



4. Logique du pipeline « UEBA Wallix »

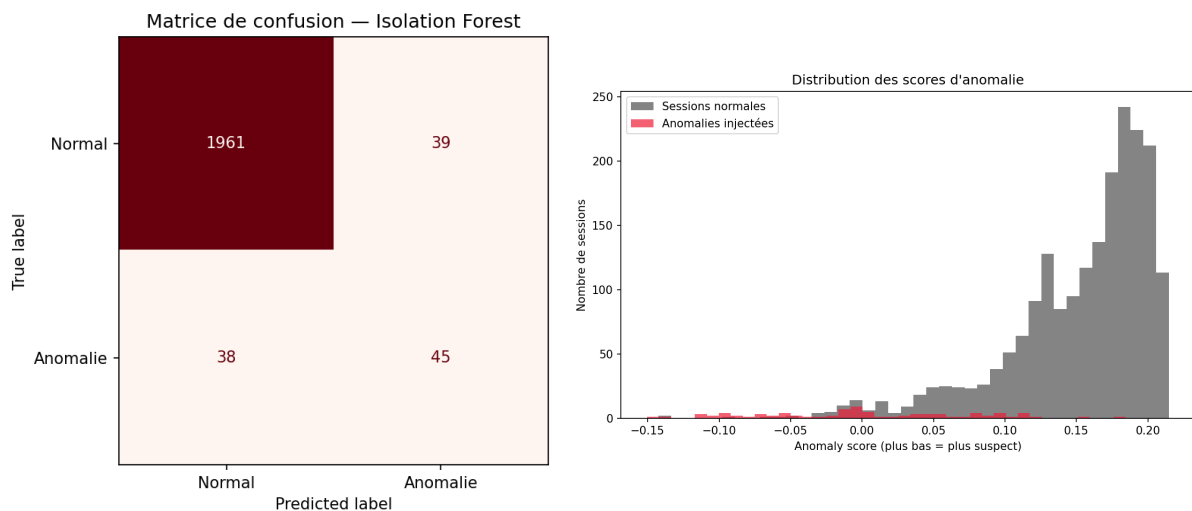
UEBA = *User and Entity Behavior Analytics* : au lieu de règles fixes, on apprend le comportement "normal" de chaque utilisateur et on signale ce qui s'en écarte. Construit de la Phase 9 à la Phase 12 (scripts/phase9_wallix_main.py → phase12) :

1. Simuler des sessions PAM pour 4 profils fixes (développeur, DBA, admin réseau, compte de service) avec des habitudes propres (heure, machines cibles), puis injecter volontairement 4% de sessions anormales (heure, machine, durée ou commande inhabituelle).
2. Transformer chaque session en variables numériques : heure encodée en cercle trigonométrique (0h et 23h doivent être "proches"), et un score de rareté d'asset propre à chaque utilisateur (est-ce que CET utilisateur va souvent sur CETTE machine ?).
3. Entraîner un **Isolation Forest** (scikit-learn), un modèle **non-supervisé** : il ne voit jamais quelles sessions sont réellement anormales pendant l'entraînement, seulement les variables numériques.
4. Comparer, après coup seulement, les prédictions du modèle aux anomalies réellement injectées, pour mesurer sa performance.

Indicateurs d'évaluation (core/wallix_evaluation.py)

Indicateur	Ce qu'il mesure
Precision	Parmi les sessions signalées suspectes, quelle proportion l'était vraiment (faux positifs)
Recall (rappel)	Parmi les vraies anomalies injectées, quelle proportion a été retrouvée (faux négatifs)
F1-score	Moyenne harmonique de la precision et du recall — résumé équilibré des deux

Illustration — résultats réels du modèle (docs/screenshots/)



5. Convergence : restitution & entrepôt de données

Les deux pipelines produisent chacun un DataFrame final (comptes scorés / sessions scorées). Trois briques indépendantes les consomment ensuite, sans jamais dupliquer la logique métier :

Brique	Ce qu'elle affiche	Déclenchée par
Dashboard Streamlit	Les deux pipelines, en direct, dans un onglet chacun	streamlit run dashboard.py
Rapport PDF	Résumé exécutif + top 10 des comptes à risque (AD uniquement)	scripts/phase8_main.py
Data Warehouse PostgreSQL	Schéma en constellation (4 dimensions + 2 tables de faits) réunissant AD et Wallix	scripts/phase13_build_datawarehouse_main.py
Power BI	Rapport visuel construit sur l'entrepôt PostgreSQL	dashboard_powerbi_ueba.pbix

Point d'attention conservé tel quel dans le code (core/db.py) : les identifiants utilisateurs AD (générés par Faker) et les profils Wallix (4 comptes fixes) ne se recoupent pas naturellement dans les données actuelles. L'entrepôt les stocke côte à côte avec une colonne source plutôt que de prétendre à une correspondance qui n'existe pas.